

Chihuahua or Muffin? With Convolutional Neural Networks.

In the second part of the “Chihuahua or Muffin” image classification series, I used Convolutional Neural Networks to classify images as either Chihuahuas or muffins. Unlike in the first part of the series, in which the CNN model uses three convolutional blocks. Each block applies a convolution layer, a ReLU activation, and max pooling. In the process, the convolutional layers learn image features based on the provided input data, which creates an output that is flattened and passed into fully connected layers for the final two-class prediction. I resized the image quality to 64 x 64 pixels which were then normalized before training, which uses random horizontal flips and small rotations as data augmentation.

CNNs are different from regular neural networking models, especially the one used in part one because it does not treat the image only as a flat list of pixel values. This is because regular neural networks, while they can learn patterns, loses much of the spatial relationship between nearby pixels when the image is flattened. CNNs, on the other hand, preserves image structure and learns local visual patterns such as edges, texture, shapes, and small combinations of features. This makes CNNs important as chihuahuas and muffins share visual similarities in color and texture, so the model needs to learn details and features such as eyes, ears, noses, muffin tops, and background patterns.

After training the CNN, I reviewed the validation accuracy and the prediction images. The validation results showed whether the model was capable of generalizing the mages it did not see during training. Correct predictions were easier when the Chihuahua face, ears, and eyes were visible or when the muffin shape was clear. However, misclassifications are a significant issue because the model tries to interpret which is which, especially when a muffin had dark spots that looked like eyes or when a Chihuahua image was small, blurry, or had similar brown colors to a muffin. This made sense because the two classes were intentionally chosen to be visually confusing. Furthermore, the model performance also depended on the training settings. Even though increasing the number of epochs can improve learning, too many epochs may cause overfitting if the validation accuracy stops improving while training accuracy continues to increase. Changing the learning rate also affected the performance of the model, as the learning rate that is too large can make training unstable, while a smaller learning rate may train more smoothly but more slowly.

When compared with a traditional neural network, the CNN is better suited for image classification because it uses convolutional filters to learn spatial features. While a traditional neural network may train faster for a small image dataset because it is simpler, it usually needs more parameters and may not understand image structure as well. Even though the CNN model

takes time to train, it is capable of achieving better validation accuracy because it extracts meaningful visual features from the image before classification. According to the lab, the CNN-based architecture was able to provide a clear definition on how CNN-based algorithms are capable of connecting model design with image recognition, and how, unlike with other models, CNN-based models are able to learn useful features directly from the training images instead of manually selecting features, making them a critical benefit of deep learning for computer vision tasks.

A challenge that I faced when dealing with CNN models is that I had to make sure that the notebook had the required important and packages installed in order for the model to work properly as intended. For example, PyTorch, torchvision, tqdm, matplotlib, and torchsummary were needed in order for the CNN model to function. Another challenge was ensuring that the data folders were in the correct structure, and this is because if the paths were wrong, then the Image Folder loader would not function properly, therefore leading to compromised results. Another challenge that I also faced was interpreting the model results. Even though accuracy can crate useful results, it is difficult to see if the model had made any mistakes. The visualization step helped because it provided transparency, which helped because from here I can look at the predicted and true labels from the results.

The capabilities of CNN-based models are useful in a wide range of real-world applications. CNN models are widely used in medical imaging, manufacturing quality inspection, self-driving vehicle perception, security screening, wildlife monitoring, agriculture, and retail image search. Despite the CNN's use in a wide range of products, the main ideas and core concepts remain the exact same, learning from labeled images and the classification of new ones.

While CNN models have been used in a wide range of applications thank to their capabilities, there are important ethical considerations when developing and deploying image classification models. First, the training data should be representative. If the data is biased or too limited, the mode may perform poorly on certain types of images. Second, to ensure transparency and to ensure high accuracy, CNNs shouldn't be used blindly in high-risk situations without human review. Lastly, and most importantly, privacy is required even when the images contain people or personal information. During the development of CNN-based models, developers should also explain model limitations clearly, even when errors could affect safety, fairness, or trust.

In conclusion, the lab helped me get a clear picture on how CNNs are widely used in computer vision. The CNN-based model learned visual features directly from images and performed classification based on those learned features. In addition, I also learned that model performance depends on the quality, training choices, evaluation, and responsible use.

Works cited

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.